

Spring Data JPA O/R Mapping Notes

Object-Relational Mapping (ORM) maps Java objects to relational database tables.

1. @ManyToOne

Represents a many-to-one relationship.

Example:

@Entity

```
class Student{
```

@ManyToOne

@JoinColumn(name="department_id")

```
private Department department;
```

```
}
```

Many students belong to one department.

2. @JoinColumn

Specifies the foreign key column used for a relationship.

Example:

@JoinColumn(name="department_id")

3. @OneToMany

Represents one parent with multiple child records.

Example:

@OneToMany(mappedBy="department", fetch=FetchType.LAZY)

```
private List students;
```

4. mappedBy

Indicates the inverse side of a bidirectional relationship.

It prevents creation of an extra join table.

Example:

@OneToMany(mappedBy="department")

5. FetchType.EAGER

Related entities are loaded immediately with the parent.

Example:

@ManyToOne(fetch=FetchType.EAGER)

Advantages:

- Simpler access.

Disadvantages:

- May load unnecessary data.

6. FetchType.LAZY

Related entities are loaded only when accessed.

Example:

@OneToMany(fetch=FetchType.LAZY)

Advantages:

- Better performance for large relationships.

Disadvantages:

- LazyInitializationException if accessed outside session.

7. @ManyToMany

Represents many records related to many records.

Example:

Student <-> Course.

@Entity

```
class Student{
```

```
@ManyToMany
```

```
@JoinTable(
```

```
name="student_course",
```

```
joinColumns=@JoinColumn(name="student_id"),
```

```
inverseJoinColumns=@JoinColumn(name="course_id"))
```

```
private List courses;
```

```
}
```

8. @JoinTable

Defines the intermediate table for many-to-many mapping.

Contains:

- joinColumns
- inverseJoinColumns

9. Relationship Summary

@ManyToOne: Many children -> One parent

@OneToMany: One parent -> Many children

@ManyToMany: Many -> Many using join table

10. Example Model

Department

id

name

Student

id

name

department_id

Course

id

title

student_course

student_id

course_id

11. Best Practices

- Use LAZY for collections.
- Use EAGER only when always required.
- Use mappedBy for bidirectional mappings.
- Use JoinTable only for many-to-many.
- Keep entity relationships simple.

12. Flow

Java Entity

->

Hibernate/JPA

->

SQL Tables

->

Database

Common annotations:

@Entity

@Id

@ManyToOne

@OneToMany

@ManyToMany

@JoinColumn

@JoinTable

mappedBy

FetchType.LAZY

FetchType.EAGER